

Objetos y referencias en memoria

Python | Etapa 1

Guía de aprendizaje autónomo

Comprende qué cambia cuando ejecutas una instrucción: el estado de un objeto, una referencia o ambos. Aprende a predecir esos cambios, explicarlos y comprobarlos.

Lee y razona	Comprueba y corrige
10 temas progresivos 20 ejercicios de predicción 10 variaciones	10 ejemplos resueltos 4 prácticas de laboratorio Respuestas razonadas

Papel primero

Lee, predice y dibuja antes de abrir el intérprete. En las prácticas señaladas, ejecuta el código y compara el resultado con tu explicación. Conserva tus errores iniciales: te ayudan a localizar qué debes revisar.

Nombre:

Grupo:

Fecha:

Edición 1.1 | Septiembre de 2026

Formato carta, preparado para impresión en blanco y negro. Los programas del estudiante funcionan con Python 3.10 o posterior y su biblioteca estándar.

TU RECORRIDO

Avanza por comprensión

Sigue el orden. Avanza cuando puedas explicar el ejemplo y resolver una variación. Si una predicción falla, revisa las referencias antes de continuar.

Tema	Página
1.1 Objetos, tipos y mutabilidad	4
1.2 Variables, asignación y referencias	8
1.3 Identidad e igualdad	11
1.4 Mutación y reasignación	14
1.5 Referencias en funciones	19
1.6 Copias de listas simples	24
1.7 Anidamiento y copia superficial	27
1.8 Repetición de listas y matrices	30
1.9 Copia profunda	35
1.10 Integración y matices de mutabilidad	38

Momentos de ejecución

Práctica	Después de	Página
P01 Identidad y cambios de estado	1.4	16
P02 Modificar, reasignar y devolver	1.5	21
P03 Construir filas independientes	1.8	32
P04 Un equipo de trabajo independiente	1.10	40

Cierre y autoevaluación: página 43. Glosario y fuentes: página 44.

Cómo leer los diagramas

Las cajas representan objetos o nombres, según su etiqueta. Las flechas representan referencias. A, B y C son etiquetas simbólicas, nunca direcciones reales. Los valores dentro de las cajas de listas se abrevian; el dibujo no representa posiciones físicas contiguas de memoria.

ANTES DE EMPEZAR

Cómo trabajar con la guía

Para empezar, necesitas reconocer variables, índices, claves de diccionarios, ciclos sencillos y la sintaxis de una función. Aquí tienes un recordatorio para consultar mientras trabajas.

Acción	Qué haces
1. Leer	Estudia el concepto, el ejemplo resuelto y su diagrama.
2. Predecir	Escribe la salida y justifica qué objeto o referencia cambia.
3. Comprobar	En las prácticas señaladas, ejecuta y registra el resultado real.
4. Investigar	Sigue las flechas y corrige la explicación, conservando tu predicción.
5. Modificar	Cambia una sola condición y predice otra vez antes de ejecutar.

Recordatorio de sintaxis

Expresión	Lectura
<code>datos[0]</code>	Primer elemento de una lista. Los índices empiezan en 0.
<code>alumno["edad"]</code>	Valor asociado a la clave "edad" de un diccionario.
<code>lista.append(x)</code>	Agrega un elemento al final de la misma lista.
<code>lista[:]</code>	Crea una copia superficial de la lista.
<code>def f(datos):</code>	Define la función <code>f()</code> con el parámetro local <code>datos</code> .
<code>return resultado</code>	Entrega un objeto al código que llamó a la función.
<code>range(3)</code>	Permite recorrer los valores 0, 1 y 2 en un ciclo.
<code>None</code>	Objeto que representa la ausencia de un resultado útil en estos ejemplos.

Preparación para el laboratorio

Abre la carpeta `etapa_01_objetos_referencias`. Ejecuta cada archivo completo, desde el inicio. En macOS o Linux: `python3 ejercicios/e05.py`. En Windows: `py ejercicios/e05.py`. También puedes usar tu editor.

No se necesitan servicios en línea ni paquetes adicionales. Mantén la guía impresa abierta y utiliza el intérprete para comprobar tus predicciones. En un cuaderno interactivo, reinicia el entorno antes de cada ejemplo para evitar estados anteriores.

Compara las identidades con respuestas `True` o `False`. No necesitas memorizar números de `id()` ni direcciones de memoria.

LECTURA Y RAZONAMIENTO

1.1 Objetos, tipos y mutabilidad

Meta: Reconocer qué objetos pueden cambiar y qué ocurre cuando una variable señala otro objeto.

Si los enteros no pueden cambiar, ¿por qué funciona `n = n + 1`?

En Python trabajamos con objetos. Cada uno tiene un tipo, un valor y una identidad. En `x = 5`, el objeto es de tipo `int` y su valor es `5`. Su identidad permite reconocer ese objeto concreto, aunque otros tengan el mismo valor.

Un objeto mutable puede cambiar su contenido; uno inmutable lo conserva. Una variable es un nombre que señala un objeto. Por eso `n = n + 1` funciona: `n` pasa a señalar el entero resultante. El entero anterior conserva su valor.

Mutables	Inmutables
<code>list</code> , <code>dict</code> , <code>set</code>	<code>int</code> , <code>float</code> , <code>bool</code> , <code>str</code> , <code>tuple</code>
Consulta adicional: <code>bytearray</code>	Consulta: <code>complex</code> , <code>bytes</code> , <code>range</code> , <code>frozenset</code>

Ejemplo resuelto D01

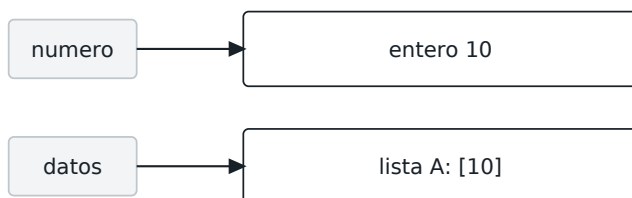
```
numero = 10
numero = numero + 1
datos = [10]
datos[0] = 11
print(numero)
print(datos)
```

Salida

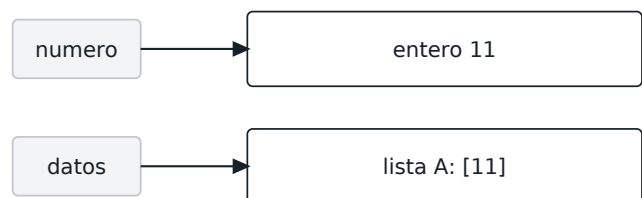
```
11
[11]
```

La suma da `11` y `numero` pasa a señalar ese entero. El entero `10` conserva su valor. En la lista, `datos[0] = 11` reemplaza su primer elemento: cambia la misma lista, no el entero que estaba allí.

Antes de las operaciones



Después de las operaciones



Observa: Que una variable muestre otro valor no demuestra que el objeto anterior haya cambiado. Por ejemplo, `upper()` obtiene una cadena en mayúsculas sin modificar la cadena original.

UNA ANALOGÍA PARA SEGUIR LAS REFERENCIAS

1.1 Tarjetas, etiquetas y flechas

Imagina tarjetas con números impresos. Una tiene el 5 y otra el 6. Como representan enteros, no podemos borrar el número para escribir otro. Cada tarjeta tiene además un folio que permite reconocerla.

La variable `x` es una etiqueta: señala una tarjeta. Si lo dibujamos, la flecha es la referencia y el folio representa la identidad. A y B son folios inventados para el dibujo; no son resultados de `id()`.

Concepto	En <code>x = 5</code>
Nombre	La etiqueta <code>x</code> .
Referencia	La flecha desde <code>x</code> hasta la tarjeta.
Tipo	<code>int</code> : la tarjeta representa un entero.
Valor	El número 5 impreso en la tarjeta.
Identidad	La tarjeta concreta, que distinguimos con su folio A.

Asignar puede llevarnos a la misma tarjeta o a otra

Con `x = x`, miramos qué tarjeta señala `x` y volvemos a colocar la etiqueta en esa tarjeta. Hay una asignación, pero la etiqueta termina donde ya estaba.

Con `x = x + 1`, leemos el 5, sumamos 1 y colocamos `x` en la tarjeta del 6. No cambiamos el número de la primera tarjeta: ahora señalamos otra.

```
x = 5
identificador_inicial = id(x)
x = x
print(id(x) == identificador_inicial)
x = x + 1
print(id(x) == identificador_inicial)
```

Salida: `True` y después `False`.

La identidad pertenece al objeto. Una asignación puede dejar una variable señalando el mismo objeto o hacer que señale otro. En este ejemplo, cuando cambia `id(x)`, cambia el objeto que señala `x`; ninguna tarjeta cambia de identidad.

RESUELVE EN PAPEL

1.1 Predice y explica

Escribe la salida antes de ejecutar. Conserva tu primera respuesta y usa cajas y flechas para justificarla. Los programas se reinician de forma independiente.

E01 | Una operación sobre una cadena

```
texto = "hola"  
mayusculas = texto.upper()  
print(texto)  
print(mayusculas)
```

1. Escribe las dos líneas de salida.
2. ¿Se modificó la cadena que señala `texto`? Explica.

Predicción, dibujo o explicación:

E02 | Modificar un diccionario

```
alumno = {"nombre": "Ana", "edad": 18}  
alumno["edad"] = 19  
print(alumno["nombre"])  
print(alumno["edad"])
```

1. Escribe la salida e identifica qué objeto cambió.
2. Clasifica `int`, `str`, `list`, `tuple`, `dict` y `set` según su mutabilidad.

Predicción, dibujo o explicación:

Una variación

En E01, reemplaza la segunda línea por `texto = texto.upper()` y muestra solo `texto`. ¿Las cadenas dejarían de ser inmutables?

Predicción de la variación y motivo:

RESPUESTAS RAZONADAS | DESPUÉS DEL INTENTO

1.1 Comprueba tu razonamiento

Compara tus respuestas con las explicaciones. Si te equivocaste, conserva tu predicción inicial e identifica qué objeto o referencia interpretaste de otra manera.

E01 | Salida esperada

```
hola  
HOLA
```

`texto.upper()` obtiene la cadena en mayúsculas. La cadena original conserva `hola`, y `texto` sigue señalándola. La variable `mayusculas` señala el resultado: `HOLA`.

E02 | Salida esperada

```
Ana  
19
```

Cambia el diccionario de `alumno`, en la entrada `"edad"`. El entero `18` no se transforma en `19`. Los tipos `int`, `str` y `tuple` son inmutables; `list`, `dict` y `set` son mutables.

La variación

Se imprimiría `HOLA`. Las cadenas siguen siendo inmutables: ahora `texto` señala la cadena resultante, y la original conserva su contenido.

Antes de avanzar

Puedo explicar la diferencia entre cambiar un objeto y hacer que una variable señale otro.

Marca tu situación: ☐ Puedo justificarlo. ☐ Necesito revisar un ejemplo.

Mi corrección o explicación con mis propias palabras:

Si la explicación aún no es clara, vuelve al diagrama del tema y sigue cada flecha desde la instrucción que produce el cambio.

LECTURA Y RAZONAMIENTO

1.2 Variables, asignación y referencias

Meta: Dibujar las referencias y reconocer cuándo varios nombres señalan el mismo objeto.

Cuando escribes `y = x`, ¿creas otra lista o das otro nombre a la misma lista?

Piensa en una variable como una etiqueta y en un objeto como una tarjeta. La referencia es la flecha que une ambos. Con `x = [1, 2, 3]` creamos una lista y la señalamos con `x`. Con `y = x`, también `y` señala esa lista.

La asignación no hace una copia. Si dos nombres señalan el mismo objeto, los llamamos alias. Cuando modificas la lista usando uno, ves el cambio también desde el otro: ambos te llevan a la misma lista.

Ejemplo resuelto D02

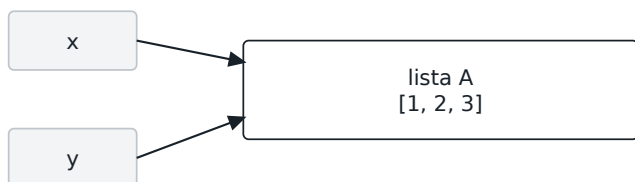
```
x = [1, 2, 3]
y = x
y.append(4)
print(x)
print(y)
```

Salida

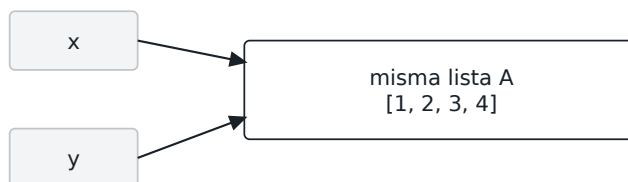
```
[1, 2, 3, 4]
[1, 2, 3, 4]
```

`x` y `y` señalan la misma lista. Al ejecutar `y.append(4)`, esa lista cambia. Las dos impresiones muestran el cambio porque consultan el mismo objeto.

Después de `y = x`



Después de `y.append(4)`



Observa: Dos nombres no significan dos objetos. Cuenta las cajas de objetos y sigue las flechas. Los folios A y B de los dibujos son simbólicos; no son direcciones reales.

RESUELVE EN PAPEL

1.2 Predice y explica

Escribe la salida antes de ejecutar. Conserva tu primera respuesta y usa cajas y flechas para justificarla. Los programas se reinician de forma independiente.

E03 | Una tercera referencia

```
a = [7]
b = a
c = b
c.append(8)
print(a)
print(b)
print(c)
```

1. Predice la salida y dibuja los tres nombres.
2. ¿Cuántas listas se crearon?

Predicción, dibujo o explicación:

E04 | La misma asignación con enteros

```
a = 7
b = a
b = b + 1
print(a)
print(b)
```

1. Escribe la salida y dibuja las referencias al final.
2. ¿Se modificó el entero 7?

Predicción, dibujo o explicación:

Una variación

En E03, cambia únicamente `b = a` por `b = [7]`. Conserva `c = b`. ¿Qué se imprimirá y cuántas listas habrá?

Predicción de la variación y motivo:

RESPUESTAS RAZONADAS | DESPUÉS DEL INTENTO

1.2 Comprueba tu razonamiento

Compara tus respuestas con las explicaciones. Si te equivocaste, conserva tu predicción inicial e identifica qué objeto o referencia interpretaste de otra manera.

E03 | Salida esperada

```
[7, 8]
[7, 8]
[7, 8]
```

Se crea una sola lista. Los nombres `a`, `b` y `c` la señalan. La instrucción `c.append(8)` la modifica; las asignaciones anteriores no hicieron copias.

E04 | Salida esperada

```
7
8
```

Después de `b = a`, ambos nombres señalan el entero `7`. La suma hace que `b` pase a señalar el `8`. La variable `a` sigue señalando el `7`, que conserva su valor.

La variación

Se imprimen `[7]`, `[7, 8]` y `[7, 8]`. La variable `a` señala una lista; `b` y `c` comparten la segunda, creada con `[7]`.

Antes de avanzar

Puedo contar los objetos sin confundirlos con los nombres que los señalan.

Marca tu situación: ☐ Puedo justificarlo. ☐ Necesito revisar un ejemplo.

Mi corrección o explicación con mis propias palabras:

Si la explicación aún no es clara, vuelve al diagrama del tema y sigue cada flecha desde la instrucción que produce el cambio.

LECTURA Y RAZONAMIENTO

1.3 Identidad e igualdad

Meta: Predecir si dos nombres señalan el mismo objeto, sin memorizar números de identificación.

¿Dos listas que imprimen lo mismo tienen que ser la misma lista?

La identidad distingue a un objeto concreto. En la analogía, es el folio de la tarjeta; la flecha representa la referencia. Para nuestras listas, `==` compara el contenido e `is` comprueba si se trata del mismo objeto. Dos listas pueden ser iguales y tener identidades distintas.

La función `id(objeto)` devuelve un número que identifica al objeto mientras existe. En CPython es su dirección de memoria; en otras implementaciones no tiene que serlo. Compararemos si los identificadores coinciden entre objetos que existen al mismo tiempo.

Ejemplo resuelto D03

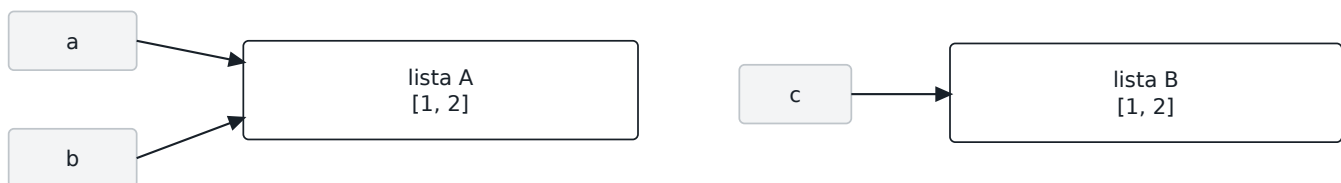
```
a = [1, 2]
b = a
c = [1, 2]
print(a == c)
print(a is c)
print(a is b)
print(id(a) == id(b))
```

Salida

```
True
False
True
True
```

`a` y `b` comparten una lista. La variable `c` señala otra con el mismo contenido. Por eso `a == c` da `True`, pero `a is c` da `False`. Los identificadores de `a` y `b` coinciden porque señalan el mismo objeto.

Dos listas distintas; `a` y `b` comparten A



Observa: Usa `==` para comparar valores. Con números y cadenas, Python puede reutilizar objetos: no saques reglas generales a partir de `is` entre literales. Cuando un objeto deja de existir, su identificador puede reutilizarse.

RESUELVE EN PAPEL

1.3 Predice y explica

Escribe la salida antes de ejecutar. Conserva tu primera respuesta y usa cajas y flechas para justificarla. Los programas se reinician de forma independiente.

E05 | Tres comparaciones

```
a = [5]
b = [5]
c = a
print(a == b)
print(a is b)
print(b is c)
print(id(a) == id(c))
```

1. Predice cada `True` o `False`; no escribas direcciones.
2. Dibuja las listas e indica qué nombres señalan el mismo objeto.

Predicción, dibujo o explicación:

E06 | Objetos vacíos

```
x = []
y = x
z = []
print(x == z)
print(x is y)
print(id(y) == id(z))
```

1. Escribe las tres líneas de salida.
2. ¿Podemos exigir un número específico como respuesta a `id(x)` en cualquier ejecución?

Predicción, dibujo o explicación:

Una variación

En E05, cambia únicamente `c = a` por `c = b`. ¿Cuáles de las cuatro respuestas cambian?

Predicción de la variación y motivo:

RESPUESTAS RAZONADAS | DESPUÉS DEL INTENTO

1.3 Comprueba tu razonamiento

Compara tus respuestas con las explicaciones. Si te equivocaste, conserva tu predicción inicial e identifica qué objeto o referencia interpretaste de otra manera.

E05 | Salida esperada

True
False
False
True

a y c comparten una lista; b señala otra. Las dos listas contienen 5, pero son objetos distintos. La última comparación da True porque a y c señalan el mismo objeto.

E06 | Salida esperada

True
True
False

Cada [] crea otra lista. Las listas de x y z están vacías y son iguales, pero son objetos distintos. La variable y comparte la de x. No se puede exigir un valor numérico fijo para id(x) entre ejecuciones.

La variación

Ahora se imprime True, False, True y False: cambian la tercera y la cuarta respuesta. En E06 no podemos exigir un número fijo para id(x); buscamos saber si dos identificadores coinciden.

Antes de avanzar

Puedo explicar cuándo dos listas son iguales y cuándo son la misma lista.

Marca tu situación: ☐ Puedo justificarlo. ☐ Necesito revisar un ejemplo.

Mi corrección o explicación con mis propias palabras:

Si la explicación aún no es clara, vuelve al diagrama del tema y sigue cada flecha desde la instrucción que produce el cambio.

LECTURA Y RAZONAMIENTO

1.4 Mutación y reasignación

Meta: Distinguir si una instrucción modifica un objeto o cambia el objeto que señala una variable.

Si `x` y `y` comparten una lista, ¿por qué `y.append(4)` y `y = y + [4]` afectan de forma distinta lo que vemos mediante `x`?

Mutar es cambiar un objeto existente. Por ejemplo, `x.append(3)` agrega un elemento a la misma lista. Asignar es indicar qué objeto señala un nombre: con `x = x` seguimos en el mismo; con `x = x + [3]` señalamos una lista nueva.

Con listas, `+` crea otra lista y `+=` amplía la existente. Con enteros, `+=` hace que el nombre señale el resultado de la suma. Antes de decidir qué cambia, mira el tipo de objeto y la operación.

Ejemplo resuelto D04

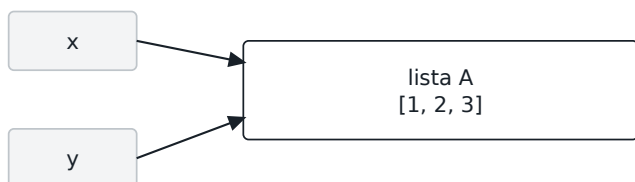
```
x = [1, 2, 3]
y = x
y = y + [4]
print(x)
print(y)
print(x is y)
```

Salida

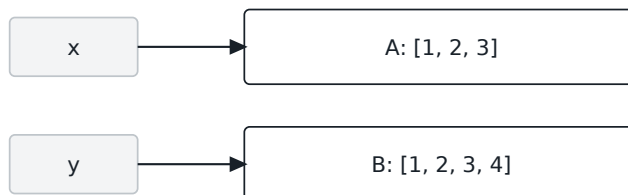
```
[1, 2, 3]
[1, 2, 3, 4]
False
```

Primero, `y + [4]` crea otra lista. Después, `y` pasa a señalarla. La variable `x` sigue señalando la lista inicial. Si hubiéramos usado `y.append(4)`, habría cambiado la lista compartida y lo veríamos también mediante `x`.

Antes de la concatenación



Después de `y = y + [4]`



Observa: `x.clear()` vacía la misma lista; `x = []` hace que `x` señale otra. Los métodos `append()` y `clear()` devuelven `None`: si escribes `x = x.append(4)`, al final `x` señalará `None`.

RESUELVE EN PAPEL

1.4 Predice y explica

Escribe la salida antes de ejecutar. Conserva tu primera respuesta y usa cajas y flechas para justificarla. Los programas se reinician de forma independiente.

E07 | La suma aumentada sobre una lista

```
x = [1, 2]
y = x
y += [3]
print(x)
print(y)
print(x is y)
```

1. Predice las tres salidas.
2. ¿Qué diferencia hay con el ejemplo D04?

Predicción, dibujo o explicación:

E08 | Vaciar y después reasignar

```
x = [1, 2]
y = x
y.clear()
y = [9]
print(x)
print(y)
print(x is y)
```

1. Dibuja las referencias después de `y.clear()` y al final.
2. ¿Qué línea modifica la lista y cuál hace que `y` señale otra?

Predicción, dibujo o explicación:

Una variación

En E07, cambia `y += [3]` por `y = y + [3]`. Después analiza `a = 10; b = a; b += 3`. ¿Qué cambia en cada caso?

Predicción de la variación y motivo:

COMPRUEBA EN PYTHON | PRÁCTICA DE LABORATORIO

P01 Identidad y cambios de estado

Meta: Comprobar qué ocurre al ampliar una lista compartida, crear otra lista y sumar a un entero.

Archivo: `practicas/p01.py`. Abre la carpeta `etapa_01_objetos_referencias`.

Programa inicial: primero predice

```
x = [1, 2]
y = x
y += [3]
a = 10
b = a
b += 3
print(x)
print(x is y)
print(a, b)
```

1. Predice la salida y dibuja las referencias del programa inicial.
2. Ejecuta el archivo. Conserva tu predicción y explica las diferencias que encuentres.
3. Variante A: cambia solo `y += [3]` por `y = y + [3]`. Predice y vuelve a ejecutar desde el inicio.
4. Variante B: haz que las listas de `x` y `y` terminen vacías y sigan siendo el mismo objeto. Elige entre `y.clear()` y `y = []`, y comprueba tu decisión.

Registro de comprobación

Puedes ampliar este registro en tu cuaderno. Cada variante se ejecuta desde el estado inicial.

Caso	Predicción	Observación y explicación
Inicial		
Variante / corrección		

Para ejecutar: `python3 practicas/p01.py` | En Windows: `py practicas/p01.py`

RESPUESTAS RAZONADAS | DESPUÉS DEL INTENTO

1.4 Comprueba tu razonamiento

Compara tus respuestas con las explicaciones. Si te equivocaste, conserva tu predicción inicial e identifica qué objeto o referencia interpretaste de otra manera.

E07 | Salida esperada

```
[1, 2, 3]
[1, 2, 3]
True
```

Con listas, `+=` agrega elementos a la misma lista. Por eso `x` y `y` siguen compartiéndola. En D04, la operación `+` creó otra lista y después `y` pasó a señalarla.

E08 | Salida esperada

```
[]
[9]
False
```

Primero, `y.clear()` vacía la lista compartida. Después, `y = [9]` crea otra lista y hace que `y` la señale. La variable `x` sigue señalando la lista vacía.

La variación

Con listas se imprime `[1, 2]`, `[1, 2, 3]` y `False`: la suma crea otra lista. Con enteros, `a` sigue señalando el `10` y `b` pasa a señalar el `13`. El entero `10` conserva su valor.

Antes de avanzar

Puedo decir qué objeto señala cada nombre antes y después de una instrucción.

Marca tu situación: ☐ Puedo justificarlo. ☐ Necesito revisar un ejemplo.

Mi corrección o explicación con mis propias palabras:

Si la explicación aún no es clara, vuelve al diagrama del tema y sigue cada flecha desde la instrucción que produce el cambio.

SOLUCIÓN DE REFERENCIA | DESPUÉS DEL INTENTO

P01 Revisión de la práctica

Inicial: se imprime `[1, 2, 3]`, `True` y `10 13`. La lista cambia; el entero `10` conserva su valor.

Variante A: se imprime `[1, 2]`, `False` y `10 13`. La operación `+` crea otra lista y `y` pasa a señalarla.

Variante B: `y.clear()` cumple ambas condiciones. Puedes imprimir `y` y comprobar `id(x) == id(y)`: debe dar `True`.

Solución de la variante B

```
x = [1, 2]
y = x
y.clear()
a = 10
b = a
b += 3
print(x)
print(x is y)
print(a, b)
```

Salida

```
[]
True
10 13
```

`clear()` vacía la lista que comparten `x` y `y`. Con `y = []`, la lista de `x` conservaría su contenido y los nombres señalarían listas distintas.

Archivo de referencia: `soluciones/s01.py`.

¿Qué cambié y qué evidencia demuestra que mi solución cumple la consigna?

LECTURA Y RAZONAMIENTO

1.5 Referencias en funciones

Meta: Explicar cuándo una función modifica un objeto compartido y cuándo entrega un resultado con `return`.

¿Por qué una función puede agregar elementos a tu lista, pero escribir `lst = []` dentro de ella no la vacía?

Al llamar a una función, sus parámetros se convierten en nombres locales para los objetos recibidos. Es como añadir una etiqueta a cada tarjeta. Python pasa argumentos por asignación: esta regla funciona igual con listas, diccionarios, enteros y cadenas.

Si la función modifica un objeto compartido, puedes ver el cambio fuera de ella. Si reasigna su parámetro, solo cambia lo que señala ese nombre local. Para entregar un resultado al código que la llamó, usa `return` y guarda ese resultado cuando lo necesites.

Ejemplo resuelto D05

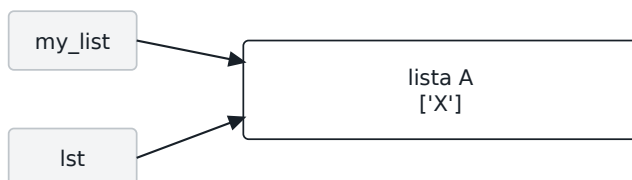
```
def add_item(lst):  
    lst.append("X")  
  
def reset_list(lst):  
    lst = []  
  
my_list = []  
add_item(my_list)  
reset_list(my_list)  
print(my_list)
```

Salida

['X']

`add_item()` agrega "X" a la lista compartida por `my_list` y `lst`. En `reset_list()`, la instrucción `lst = []` hace que solo el parámetro local señale otra lista. La variable `my_list` sigue señalando la primera.

Al entrar en `reset_list`



Durante `reset_list`, tras `lst = []`



Observa: Las listas y los enteros siguen la misma regla al entrar a una función. La diferencia está en qué operaciones permiten. Si una función termina sin ejecutar `return` con un valor, devuelve `None`.

RESUELVE EN PAPEL

1.5 Predice y explica

Escribe la salida antes de ejecutar. Conserva tu primera respuesta y usa cajas y flechas para justificarla. Los programas se reinician de forma independiente.

E09 | Reasignar dentro de la función

```
def modify(lst):  
    lst = lst + [99]  
  
numbers = [1, 2, 3]  
modify(numbers)  
print(numbers)
```

1. Escribe la salida y dibuja las dos listas durante la llamada.
2. ¿Qué cambiaría si la función usara `lst.append(99)`?

Predicción, dibujo o explicación:

E10 | Actualizar y reiniciar un diccionario

```
def update_dict(d):  
    d["key"] = "value"  
  
def reset_dict(d):  
    d = {}  
  
my_dict = {}  
update_dict(my_dict)  
reset_dict(my_dict)  
print(my_dict)
```

1. Identifica la línea que modifica el diccionario y la que reasigna `d`.
2. Propón una línea para que `reset_dict()` vacíe el diccionario recibido.

Predicción, dibujo o explicación:

Una variación

En E09, haz que la función devuelva `lst + [99]`. Compara `modify(numbers)` con `numbers = modify(numbers)`: ¿en cuál guardas la lista resultante?

Predicción de la variación y motivo:

COMPRUEBA EN PYTHON | PRÁCTICA DE LABORATORIO

P02 Modificar, reasignar y devolver

Meta: Decidir si una función debe modificar la lista recibida o devolver otra lista.

Archivo: `practicas/p02.py`. Abre la carpeta `etapa_01_objetos_referencias`.

Programa inicial: primero predice

```
def agregar_final(datos):  
    datos = datos + [99]  
  
original = [1, 2]  
resultado = agregar_final(original)  
print(original)  
print(resultado)
```

1. Predice lo que se imprime mediante `original` y `resultado`. Dibuja qué señalan `original` y `datos` dentro de la función.
2. Variante A: reemplaza el cuerpo por `datos.append(99)`, sin añadir `return`. Vuelve a predecir las salidas.
3. Variante B: conserva la lista de `original` y devuelve otra con `99` al final. Guárdala en `resultado` y compara ambas identidades.
4. En la variante B, llama a `agregar_final(original)` sin guardar el resultado. ¿Qué lista puedes seguir consultando mediante `original`?

Registro de comprobación

Puedes ampliar este registro en tu cuaderno. Cada variante se ejecuta desde el estado inicial.

Caso	Predicción	Observación y explicación
Inicial		
Variante / corrección		

Para ejecutar: `python3 practicas/p02.py` | En Windows: `py practicas/p02.py`

RESPUESTAS RAZONADAS | DESPUÉS DEL INTENTO

1.5 Comprueba tu razonamiento

Compara tus respuestas con las explicaciones. Si te equivocaste, conserva tu predicción inicial e identifica qué objeto o referencia interpretaste de otra manera.

E09 | Salida esperada

```
[1, 2, 3]
```

`lst + [99]` crea otra lista, y solo el parámetro `lst` pasa a señalarla. La variable `numbers` conserva la lista inicial. Con `lst.append(99)` sí cambiaría esa lista compartida.

E10 | Salida esperada

```
{'key': 'value'}
```

`update_dict()` modifica el diccionario de `my_dict`. La función `reset_dict()` solo hace que su parámetro `d` señale otro diccionario. Para vaciar el recibido, podría ejecutar `d.clear()`.

La variación

Sin guardar el resultado, `numbers` conserva `[1, 2, 3]`. Al asignarlo, señala `[1, 2, 3, 99]`. Si usas `lst.append(99)`, cambia la lista original. En E10, `d.clear()` vacía el diccionario recibido.

Antes de avanzar

Puedo explicar qué cambia fuera de una función y cuándo necesito guardar lo que devuelve.

Marca tu situación: ☐ Puedo justificarlo. ☐ Necesito revisar un ejemplo.

Mi corrección o explicación con mis propias palabras:

Si la explicación aún no es clara, vuelve al diagrama del tema y sigue cada flecha desde la instrucción que produce el cambio.

SOLUCIÓN DE REFERENCIA | DESPUÉS DEL INTENTO

P02 Revisión de la práctica

Inicial: se imprime `[1, 2]` y `None`. Dentro de la función, `datos` termina señalando otra lista, `[1, 2, 99]`, mientras `original` conserva la primera.

Variante A: la lista de `original` cambia a `[1, 2, 99]`. La variable `resultado` sigue recibiendo `None`: la función termina sin devolver un valor.

Variante B: la función devuelve otra lista. Si no guardas ese resultado, `original` sigue señalando `[1, 2]` y no conservas un nombre para la lista nueva.

Solución de la variante B

```
def agregar_final(datos):  
    return datos + [99]  
  
original = [1, 2]  
resultado = agregar_final(original)  
print(original)  
print(resultado)  
print(original is resultado)
```

Salida

```
[1, 2]  
[1, 2, 99]  
False
```

`return` entrega la lista nueva al código que llamó a la función. La variable `resultado` la señala, mientras `original` conserva la primera lista.

Archivo de referencia: `soluciones/s02.py`.

¿Qué cambié y qué evidencia demuestra que mi solución cumple la consigna?

LECTURA Y RAZONAMIENTO

1.6 Copias de listas simples

Meta: Elegir entre compartir una lista o copiarla para modificarla por separado.

¿Qué cambia si escribes `copia = original.copy()` en lugar de `copia = original`?

Con `original.copy()` creas otra lista. Sus posiciones empiezan señalando los mismos elementos que la original. A esto lo llamamos copia superficial: copias la lista exterior, pero no haces nuevas copias de cada objeto que contiene.

También puedes copiar una lista con `original[:]`. Si contiene enteros, cadenas o booleanos, cambiar una posición de la copia no modifica la lista original. Lo que cambia es qué objeto ocupa esa posición de la lista nueva.

Ejemplo resuelto D06

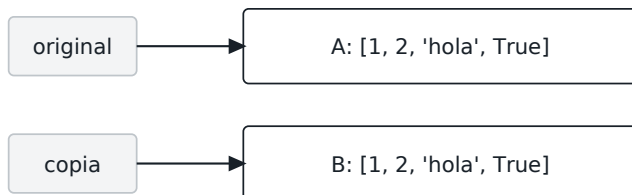
```
original = [1, 2, "hola", True]
copia = original.copy()
copia[0] = 99
print(original)
print(copia)
print(original is copia)
```

Salida

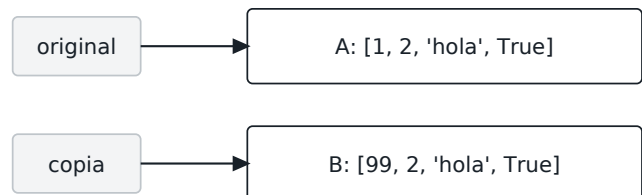
```
[1, 2, 'hola', True]
[99, 2, 'hola', True]
False
```

Hay dos listas. Reemplazar la primera posición de `copia` no cambia la de `original`. Los demás elementos pueden seguir siendo los mismos objetos inmutables, aunque las listas que los contienen sean distintas.

Después de `.copy()`



Después de `copia[0] = 99`



Observa: Llamar `copia` a una variable no crea una copia: `copia = original` comparte la lista. Más adelante veremos por qué una tupla que contiene una lista requiere mirar un nivel adicional.

RESUELVE EN PAPEL

1.6 Predice y explica

Escribe la salida antes de ejecutar. Conserva tu primera respuesta y usa cajas y flechas para justificarla. Los programas se reinician de forma independiente.

E11 | Una copia y un alias

```
original = [10, 20]
copia = original.copy()
alias = original
copia.append(30)
alias[0] = 99
print(original)
print(copia)
```

1. Predice la salida y dibuja las dos listas.
2. ¿Qué nombres señalan la misma lista?

Predicción, dibujo o explicación:

E12 | Copiar mediante un corte

```
a = ["A", "B"]
b = a[:]
print(a == b)
print(a is b)
b.clear()
print(a)
print(b)
```

1. Escribe cada línea de salida.
2. ¿Por qué vaciar la lista de `b` no vacía la de `a`?

Predicción, dibujo o explicación:

Una variación

En E12, cambia `b = a[:]` por `b = a`. Vuelve a predecir las cuatro salidas.

Predicción de la variación y motivo:

RESPUESTAS RAZONADAS | DESPUÉS DEL INTENTO

1.6 Comprueba tu razonamiento

Compara tus respuestas con las explicaciones. Si te equivocaste, conserva tu predicción inicial e identifica qué objeto o referencia interpretaste de otra manera.

E11 | Salida esperada

```
[99, 20]  
[10, 20, 30]
```

`copia` señala otra lista, a la que se agrega 30. La variable `alias` comparte la lista de `original` y reemplaza su primer elemento por 99. Son dos cambios sobre dos listas distintas.

E12 | Salida esperada

```
True  
False  
['A', 'B']  
[]
```

`a[:]` crea otra lista con los mismos elementos. Al principio son iguales, pero `a is b` da `False`. Después, `b.clear()` vacía solamente la lista de `b`.

La variación

Se imprime `True`, `True`, `[]` y `[]`. Ahora `a` y `b` comparten una lista y `b.clear()` la vacía. En E11, quienes comparten la misma lista son `original` y `alias`.

Antes de avanzar

Puedo decidir cuándo compartir una lista y cuándo necesito una copia.

Marca tu situación: ☐ Puedo justificarlo. ☐ Necesito revisar un ejemplo.

Mi corrección o explicación con mis propias palabras:

Si la explicación aún no es clara, vuelve al diagrama del tema y sigue cada flecha desde la instrucción que produce el cambio.

LECTURA Y RAZONAMIENTO

1.7 Anidamiento y copia superficial

Meta: Seguir las referencias de una lista anidada y distinguir entre reemplazar una fila y modificarla.

Si copias una lista, ¿pueden seguir compartidas las listas que hay dentro?

Una lista puede señalar otras listas. En `original[0][0]`, el primer índice nos lleva a la primera lista interior y el segundo a su primer elemento. Dibuja la lista exterior y cada lista interior en cajas separadas.

Una copia superficial crea otra lista exterior, pero conserva las referencias a las interiores. Así, `copia[0] = [99]` reemplaza una fila de la copia; `copia[0][0] = 99` cambia una fila que puede seguir compartida con `original`.

Ejemplo resuelto D07

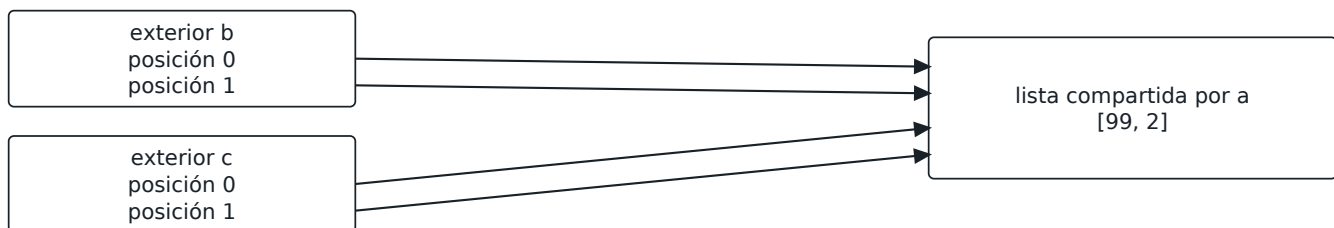
```
a = [1, 2]
b = [a, a]
c = b.copy()
c[0][0] = 99
print(a)
print(b)
print(c)
print(b is c)
```

Salida

```
[99, 2]
[[99, 2], [99, 2]]
[[99, 2], [99, 2]]
False
```

`b` y `c` son dos listas exteriores distintas. Las cuatro posiciones entre ambas señalan la misma lista de `a`. La instrucción `c[0][0] = 99` cambia esa lista interior y el cambio se ve al imprimir cualquiera de las dos listas exteriores.

Tras `c[0][0] = 99`: exteriores distintos, una lista interior



Observa: Cuando leas «se hizo una copia», pregunta: ¿de qué objeto? Después identifica cuáles siguen compartidos.

RESUELVE EN PAPEL

1.7 Predice y explica

Escribe la salida antes de ejecutar. Conserva tu primera respuesta y usa cajas y flechas para justificarla. Los programas se reinician de forma independiente.

E13 | Sustituir una fila de la copia

```
original = [[1, 2], [3, 4]]
copia = original.copy()
copia[0] = [99, 2]
print(original)
print(copia)
print(original[1] is copia[1])
```

1. Escribe la salida y señala qué referencia se reemplazó.
2. ¿Sigue compartida alguna fila?

Predicción, dibujo o explicación:

E14 | Modificar una fila compartida

```
original = [[1, 2], [3, 4]]
copia = original.copy()
copia[0][0] = 99
print(original)
print(copia)
print(original[0] is copia[0])
```

1. Predice las salidas y dibuja ambos niveles.
2. ¿Por qué E13 y E14 afectan de forma distinta a `original`?

Predicción, dibujo o explicación:

Una variación

En E14, cambia únicamente `copia[0][0] = 99` por `copia.append([5, 6])`. ¿Qué lista se modifica?

Predicción de la variación y motivo:

RESPUESTAS RAZONADAS | DESPUÉS DEL INTENTO

1.7 Comprueba tu razonamiento

Compara tus respuestas con las explicaciones. Si te equivocaste, conserva tu predicción inicial e identifica qué objeto o referencia interpretaste de otra manera.

E13 | Salida esperada

```
[[1, 2], [3, 4]]  
[[99, 2], [3, 4]]  
True
```

`copia[0] = [99, 2]` reemplaza la primera fila de la copia. La primera fila de `original` conserva su contenido. La segunda fila sigue compartida entre ambas listas exteriores.

E14 | Salida esperada

```
[[99, 2], [3, 4]]  
[[99, 2], [3, 4]]  
True
```

La primera fila sigue siendo una sola lista compartida. Al cambiar su primer elemento, ves el resultado desde `original` y desde `copia`. En E13 se reemplazó la referencia a la fila; aquí se modifica la fila misma.

La variación

Solo cambia la lista exterior `copia`. La lista `original` conserva `[[1, 2], [3, 4]]`; la copia queda `[[1, 2], [3, 4], [5, 6]]`. La primera fila sigue compartida: la comparación da `True`.

Antes de avanzar

Puedo seguir los índices hasta la lista que cambia y reconocer qué partes siguen compartidas.

Marca tu situación: ☐ Puedo justificarlo. ☐ Necesito revisar un ejemplo.

Mi corrección o explicación con mis propias palabras:

Si la explicación aún no es clara, vuelve al diagrama del tema y sigue cada flecha desde la instrucción que produce el cambio.

LECTURA Y RAZONAMIENTO

1.8 Repetición de listas y matrices

Meta: Reconocer qué referencias se repiten con `*` y construir matrices cuyas filas puedan cambiar por separado.

¿Por qué modificar una sola celda puede cambiar lo que vemos en las tres filas?

Al repetir una lista con `*`, se repiten las referencias a sus elementos. En `[0] * 3` obtienes tres posiciones que señalan el entero `0`. Puedes reemplazar una posición de esa lista sin modificar el entero.

En `[[0] * 3] * 3` hay una sola fila `[0, 0, 0]`: la lista exterior la señala tres veces. Para tener filas independientes, crea una nueva en cada vuelta del ciclo. La expresión `[[0] * 3 for _ in range(3)]` hace justamente eso.

Ejemplo resuelto D08

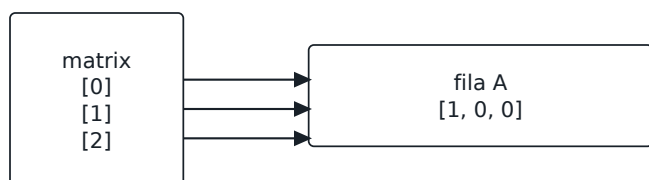
```
matrix = [[0] * 3] * 3
matrix[0][0] = 1
print(matrix)
print(matrix[0] is matrix[1])
```

Salida

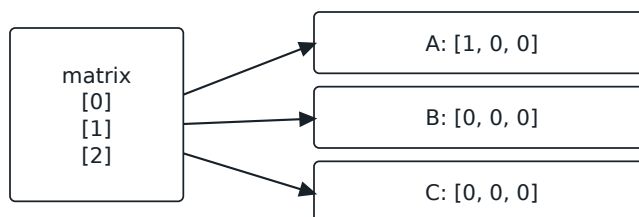
```
[[1, 0, 0], [1, 0, 0], [1, 0, 0]]
True
```

Hay una lista exterior y una sola fila interior. La instrucción `matrix[0][0] = 1` cambia esa fila. Como las tres posiciones de la lista exterior señalan la misma fila, el cambio aparece tres veces al imprimir.

Con repetición: una fila compartida



Con filas construidas por separado



Observa: La comprensión evalúa `[0] * 3` en cada vuelta. Usamos `_` porque no necesitamos el contador. Con `a = [0]` y `[a * 3] * 3`, también se crea una sola fila nueva y se comparte tres veces.

RESUELVE EN PAPEL

1.8 Predice y explica

Escribe la salida antes de ejecutar. Conserva tu primera respuesta y usa cajas y flechas para justificarla. Los programas se reinician de forma independiente.

E15 | Una variable explícita

```
a = [0]
matrix = [a * 3] * 3
matrix[0][0] = 1
print(a)
print(matrix)
print(matrix[0] is matrix[2])
```

1. Predice las tres salidas.
2. ¿La lista de `a` y la fila creada por `a * 3` son el mismo objeto? Explica.

Predicción, dibujo o explicación:

E16 | Construcción con un ciclo

```
matrix = []
for _ in range(3):
    matrix.append([0] * 3)
matrix[0][0] = 1
print(matrix)
print(matrix[0] is matrix[1])
```

1. Predice la salida y dibuja las tres filas.
2. Escribe la construcción con una comprensión de listas.

Predicción, dibujo o explicación:

Una variación

En E15, usa `matrix = [(a * 3).copy() for _ in range(3)]`. ¿Hace falta `.copy()` aquí? Con `a = [0]`, ¿qué tamaño tendría `[a.copy(), a.copy(), a.copy()]`?

Predicción de la variación y motivo:

COMPRUEBA EN PYTHON | PRÁCTICA DE LABORATORIO

P03 Construir filas independientes

Meta: Separar dos decisiones: crear filas independientes y copiar esas filas para otra matriz.

Archivo: `practicas/p03.py`. Abre la carpeta `etapa_01_objetos_referencias`.

Programa inicial: primero predice

```
matrix = [[0] * 3] * 3
copia = matrix.copy()
copia[0][0] = 1
print(matrix)
print(copia)
print(matrix is copia)
print(matrix[0] is copia[0])
```

1. Dibuja las listas exteriores y sus filas. Predice las cuatro salidas y después ejecuta.
2. Variante A: construye `matrix` con un ciclo que cree una fila en cada vuelta. Conserva `copia = matrix.copy()`. ¿Qué filas siguen compartidas?
3. Variante B: además de crear filas independientes, copia cada fila para que cambiar una celda de `copia` no cambie `matrix`. Usa lo aprendido hasta 1.8.
4. Comprueba con `is` si las filas son distintas entre sí y si cada original es distinta de su copia. Compara identidades; no escribas números fijos de `id()`.

Registro de comprobación

Puedes ampliar este registro en tu cuaderno. Cada variante se ejecuta desde el estado inicial.

Caso	Predicción	Observación y explicación
Inicial		
Variante / corrección		

Para ejecutar: `python3 practicas/p03.py` | En Windows: `py practicas/p03.py`

RESPUESTAS RAZONADAS | DESPUÉS DEL INTENTO

1.8 Comprueba tu razonamiento

Compara tus respuestas con las explicaciones. Si te equivocaste, conserva tu predicción inicial e identifica qué objeto o referencia interpretaste de otra manera.

E15 | Salida esperada

```
[0]
[[1, 0, 0], [1, 0, 0], [1, 0, 0]]
True
```

`a * 3` crea una fila distinta de la lista de `a`. La lista exterior la señala tres veces. Cambia esa fila, mientras `a` conserva `[0]`. Al final, desde `a` y `matrix` puedes llegar a tres listas: la de `a`, la fila y la exterior.

E16 | Salida esperada

```
[[1, 0, 0], [0, 0, 0], [0, 0, 0]]
False
```

Cada vuelta del ciclo crea otra fila. Al cambiar una celda de la primera, las otras dos conservan su contenido. Puedes escribir la construcción como `matrix = [[0] * 3 for _ in range(3)]`.

La variación

Las filas son independientes. Aquí sobra `.copy()`, porque `a * 3` ya crea otra fila en cada vuelta. En E16 puedes usar `[[0] * 3 for _ in range(3)]`. Si copias tres veces `a = [0]`, obtienes 3 filas y 1 columna.

Antes de avanzar

Puedo explicar por qué repetir una fila no equivale a crear varias filas independientes.

Marca tu situación: ☐ Puedo justificarlo. ☐ Necesito revisar un ejemplo.

Mi corrección o explicación con mis propias palabras:

Si la explicación aún no es clara, vuelve al diagrama del tema y sigue cada flecha desde la instrucción que produce el cambio.

SOLUCIÓN DE REFERENCIA | DESPUÉS DEL INTENTO

P03 Revisión de la práctica

Inicial: ambas matrices muestran tres veces `[1, 0, 0]`. Las listas exteriores son distintas, pero todas sus posiciones señalan la misma fila.

Variante A: `matrix` y `copia` muestran `[[1, 0, 0], [0, 0, 0], [0, 0, 0]]`. Las filas de `matrix` son distintas entre sí, pero cada una sigue compartida con su posición en `copia`.

Variante B: se separan las listas exteriores y cada fila. Para estas celdas enteras, una copia de cada fila es suficiente. Si hubiera otros objetos mutables dentro, revisaríamos ese nivel también.

Solución de la variante B

```
matrix = [[0] * 3 for _ in range(3)]
copia = [fila.copy() for fila in matrix]
copia[0][0] = 1
print(matrix)
print(copia)
print(copia[0] is copia[1])
print(matrix[0] is copia[0])
```

Salida

```
[[0, 0, 0], [0, 0, 0], [0, 0, 0]]
[[1, 0, 0], [0, 0, 0], [0, 0, 0]]
False
False
```

Cada fila se crea por separado y después se copia por separado. Como las celdas contienen enteros, basta con copiar estos dos niveles para cambiarlas sin afectar la matriz original.

Archivo de referencia: `soluciones/s03.py`.

¿Qué cambié y qué evidencia demuestra que mi solución cumple la consigna?

LECTURA Y RAZONAMIENTO

1.9 Copia profunda

Meta: Decidir qué necesitas copiar para modificar los objetos interiores sin afectar los originales.

Si necesito modificar listas interiores sin afectar al original, ¿qué debo copiar?

El módulo `copy` viene incluido en Python. Con `copy.copy(objeto)` haces una copia superficial. Con `copy.deepcopy(objeto)` también copias los contenedores interiores de nuestros ejemplos: las listas y los diccionarios anidados.

Elige según lo que necesites cambiar. Una asignación permite compartir el objeto. Una copia superficial separa el contenedor exterior. Una copia profunda permite cambiar los contenedores interiores de estos ejemplos sin modificar los originales.

Ejemplo resuelto D09

```
import copy

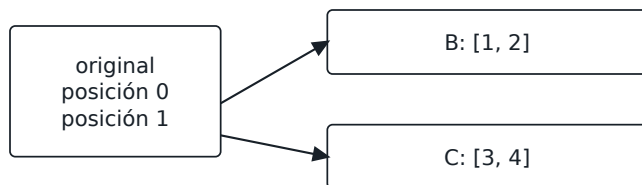
original = [[1, 2], [3, 4]]
profunda = copy.deepcopy(original)
profunda[0][0] = 99
print(original)
print(profunda)
print(original[0] is profunda[0])
```

Salida

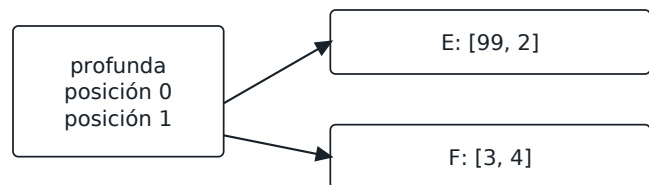
```
[[1, 2], [3, 4]]
[[99, 2], [3, 4]]
False
```

Se copian la lista exterior y las filas. Ahora `profunda[0]` y `original[0]` son listas distintas: cambiar una no modifica la otra. Para lograrlo no hace falta copiar también cada entero.

Original



Copia profunda, tras cambiar una celda



Observa: `deepcopy()` puede mantener referencias compartidas dentro de la copia. Si dos posiciones señalaban una sola fila, pueden seguir señalando una sola fila copiada. Para separar esas filas, revisa cómo las construyes.

RESUELVE EN PAPEL

1.9 Predice y explica

Escribe la salida antes de ejecutar. Conserva tu primera respuesta y usa cajas y flechas para justificarla. Los programas se reinician de forma independiente.

E17 | Una lista dentro de un diccionario

```
import copy

original = {"notas": [8, 9]}
copia = copy.deepcopy(original)
copia["notas"].append(10)
print(original["notas"])
print(copia["notas"])
```

1. Predice las dos líneas de salida.
2. ¿Qué objetos mutables necesitas copiar para que este cambio no afecte al original?

Predicción, dibujo o explicación:

E18 | Una copia profunda conserva un alias interno

```
import copy

fila = [0]
original = [fila, fila]
copia = copy.deepcopy(original)
copia[0][0] = 7
print(original)
print(copia)
print(copia[0] is copia[1])
print(copia[0] is fila)
```

1. Predice la salida y dibuja las listas originales y las copiadas.
2. ¿Las filas de `copia` son independientes de la original? ¿Son independientes entre sí?

Predicción, dibujo o explicación:

Una variación

En E17, usa `original.copy()`. En E18, usa `copia = [elemento.copy() for elemento in original]`. ¿Qué cambia en cada caso?

Predicción de la variación y motivo:

RESPUESTAS RAZONADAS | DESPUÉS DEL INTENTO

1.9 Comprueba tu razonamiento

Compara tus respuestas con las explicaciones. Si te equivocaste, conserva tu predicción inicial e identifica qué objeto o referencia interpretaste de otra manera.

E17 | Salida esperada

```
[8, 9]
[8, 9, 10]
```

Se copian el diccionario y su lista "notas". La instrucción `append(10)` cambia solo la lista copiada. Una copia superficial del diccionario dejaría esa lista compartida.

E18 | Salida esperada

```
[[0], [0]]
[[7], [7]]
True
False
```

`deepcopy()` copia la fila una vez y usa esa misma copia en las dos posiciones. La fila original conserva `[0]`. Hay independencia respecto al original, pero las dos posiciones de `copia` siguen compartiendo una fila.

La variación

En E17 se imprime `[8, 9, 10]` dos veces: la lista interior sigue compartida. En E18 se imprime `[[0], [0]]`, `[[7], [0]]`, `False` y `False`. Cada vuelta copia la fila por separado; sus elementos son enteros.

Antes de avanzar

Puedo elegir qué copiar y explicar si la copia comparte objetos con el original o entre sus propias posiciones.

Marca tu situación: ☐ Puedo justificarlo. ☐ Necesito revisar un ejemplo.

Mi corrección o explicación con mis propias palabras:

Si la explicación aún no es clara, vuelve al diagrama del tema y sigue cada flecha desde la instrucción que produce el cambio.

LECTURA Y RAZONAMIENTO

1.10 Integración y matices de mutabilidad

Meta: Resolver ejemplos que combinan funciones y contenedores, sin perder de vista qué objeto cambia.

¿Puede cambiar una lista contenida en una tupla aunque la tupla sea inmutable?

Una tupla conserva los objetos a los que apuntan sus posiciones: no puedes reemplazarlos. Pero, si uno de ellos es una lista, esa lista sí puede cambiar. Por eso puedes observar contenido diferente al imprimir la tupla sin haber reemplazado ninguno de sus elementos.

Sigue la ruta hasta el objeto que cambia. En `carro["motor"]["caballos"] = 300` modificas el diccionario del motor. Copiar solo el diccionario del carro no separa ese motor. Aquí representamos carros, personas y equipos mediante diccionarios.

Ejemplo resuelto D10

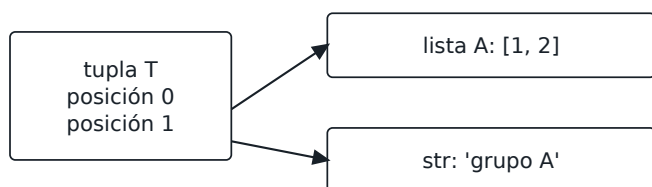
```
datos = ([1, 2], "grupo A")
alias = datos
datos[0].append(3)
print(datos)
print(alias is datos)
```

Salida

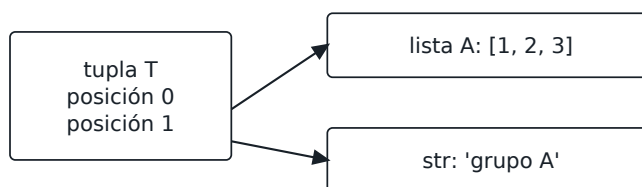
```
([1, 2, 3], 'grupo A')
True
```

La tupla sigue señalando la misma lista y la misma cadena. Solo cambia la lista interior. Si intentaras `datos[0] = [9]`, tratarías de reemplazar un elemento de la tupla y obtendrías `TypeError`.

Antes de `append(3)`



Después: mismas referencias en T



Observa: Que haya una tupla no vuelve inmutables los objetos que contiene. Que haya una copia tampoco garantiza que todos los objetos interiores sean independientes.

RESUELVE EN PAPEL

1.10 Predice y explica

Escribe la salida antes de ejecutar. Conserva tu primera respuesta y usa cajas y flechas para justificarla. Los programas se reinician de forma independiente.

E19 | Copiar una lista que contiene una tupla

```
original = [[1], "A"]
copia = original.copy()
copia[0][0].append(2)
print(original)
print(copia)
print(original[0] is copia[0])
```

1. Predice las salidas y dibuja los tres niveles.
2. ¿Qué objeto cambia y cuáles siguen compartidos?

Predicción, dibujo o explicación:

E20 | Dos carros comparten motor

```
motor = {"caballos": 200}
carro1 = {"marca": "Toyota", "motor": motor}
carro2 = carro1.copy()
carro2["marca"] = "Honda"
carro2["motor"]["caballos"] = 300
print(carro1["marca"])
print(carro1["motor"]["caballos"])
print(carro1["motor"] is carro2["motor"])
```

1. Predice las salidas. ¿Por qué la marca y el motor se comportan de forma distinta?
2. ¿Cómo harías que el motor de `carro1` conserve su valor?

Predicción, dibujo o explicación:

Una variación

En E20, importa `copy` y usa `copy.deepcopy(carro1)`. En D10, compara `datos[0].append(3)` con `datos[0] = [3]`: ¿qué objeto intenta modificar cada operación?

Predicción de la variación y motivo:

COMPRUEBA EN PYTHON | PRÁCTICA DE LABORATORIO

P04 Un equipo de trabajo independiente

Meta: Usar funciones y copia profunda para modificar un equipo sin alterar los datos del original.

Archivo: practicas/p04.py. Abre la carpeta etapa_01_objetos_referencias.

Programa inicial: primero predice

```
def preparar_equipo(equipo):
    nuevo = equipo.copy()
    nuevo["nombre"] = "Pruebas"
    nuevo["miembros"][0]["edad"] = 99
    return nuevo

ana = {"nombre": "Ana", "edad": 25}
original = {"nombre": "Desarrollo", "miembros": [ana]}
copia = preparar_equipo(original)
print(original["nombre"], ana["edad"])
print(copia["nombre"], copia["miembros"][0]["edad"])
```

- 1. Antes de ejecutar, dibuja el equipo, la lista "miembros" y el diccionario de Ana. Predice las dos salidas.
- 2. Corrige preparar_equipo() para que el equipo devuelto cambie de nombre y edad sin modificar los objetos de original y ana.
- 3. Compara con is tres niveles: equipo, lista de miembros y primer diccionario de persona. Explica cada resultado.
- 4. Después, cambia la edad de Ana en el original. Comprueba que la copia conserva su edad y explica qué demuestra.

Registro de comprobación

Puedes ampliar este registro en tu cuaderno. Cada variante se ejecuta desde el estado inicial.

Caso	Predicción	Observación y explicación
Inicial		
Variante / corrección		

Para ejecutar: python3 practicas/p04.py | En Windows: py practicas/p04.py

RESPUESTAS RAZONADAS | DESPUÉS DEL INTENTO

1.10 Comprueba tu razonamiento

Compara tus respuestas con las explicaciones. Si te equivocaste, conserva tu predicción inicial e identifica qué objeto o referencia interpretaste de otra manera.

E19 | Salida esperada

```
[[[1, 2], 'A']]  
[[[1, 2], 'A']]  
True
```

Se copia la lista exterior, pero su elemento sigue siendo la misma tupla. Esa tupla señala la misma lista interior, a la que se agrega 2. Continúan compartidas la tupla y la lista que contiene.

E20 | Salida esperada

```
Toyota  
300  
True
```

Los carros son diccionarios distintos: cambiar `carro2["marca"]` no afecta a `carro1`. Sin embargo, comparten el diccionario `motor`, que ahora contiene 300 caballos. Para conservar el motor original, puedes usar `copy.deepcopy(carro1)`.

La variación

E20 imprime `Toyota`, `200` y `False`: la copia tiene su propio diccionario del motor. En D10, `append(3)` cambia la lista interior; `datos[0] = [3]` intenta reemplazar un elemento de la tupla y produce `TypeError`.

Antes de avanzar

Puedo localizar y corregir un cambio compartido inesperado explicando cada paso.

Marca tu situación: ☐ Puedo justificarlo. ☐ Necesito revisar un ejemplo.

Mi corrección o explicación con mis propias palabras:

SOLUCIÓN DE REFERENCIA | DESPUÉS DEL INTENTO

P04 Revisión de la práctica

Inicial: se imprime Desarrollo 99 y Pruebas 99. Solo se copió el diccionario exterior. El nombre del equipo cambia por separado, pero la edad pertenece a un diccionario compartido.

Con la solución se imprime Desarrollo 25 y Pruebas 99. Las tres comparaciones dan False: se copiaron el equipo, la lista de miembros y el diccionario de la persona.

Si después escribes `ana["edad"] = 26`, el equipo original muestra 26 y la copia conserva 99. Los dos equipos tienen diccionarios de persona independientes.

Una solución con copia profunda

```
import copy

def preparar_equipo(equipo):
    nuevo = copy.deepcopy(equipo)
    nuevo["nombre"] = "Pruebas"
    nuevo["miembros"][0]["edad"] = 99
    return nuevo

ana = {"nombre": "Ana", "edad": 25}
original = {"nombre": "Desarrollo", "miembros": [ana]}
copia = preparar_equipo(original)
print(original["nombre"], ana["edad"])
print(copia["nombre"], copia["miembros"][0]["edad"])
print(original is copia)
print(original["miembros"] is copia["miembros"])
print(ana is copia["miembros"][0])
```

Salida

```
Desarrollo 25
Pruebas 99
False
False
False
```

`deepcopy()` separa los tres niveles mutables. La función devuelve el nuevo diccionario y la variable `copia` lo conserva.

Archivo de referencia: `soluciones/s04.py`.

¿Qué cambié y qué evidencia demuestra que mi solución cumple la consigna?

CIERRE DE LA ETAPA

Demuestra tu comprensión

Vuelve a un ejercicio que inicialmente resolviste mal. Sin mirar su respuesta, explica ahora el recorrido de sus referencias y el objeto afectado por cada instrucción.

Puedo...	Sí / Revisar
Distinguir objetos mutables e inmutables y explicar una reasignación.	
Anticipar coincidencias de identidad sin usar direcciones fijas.	
Separar la mutación de una lista de la reasignación de un nombre.	
Explicar los efectos de una función sobre el objeto recibido.	
Seguir referencias por varios niveles de contenedores.	
Construir filas independientes y comprobarlo.	
Elegir una copia superficial o profunda según el cambio previsto.	
Explicar una lista mutable dentro de una tupla inmutable.	

Una explicación completa contiene

1. Los objetos relevantes y los nombres o posiciones que llegan a ellos.
2. La instrucción que produce el cambio.
3. El objeto o referencia que se modifica.
4. El motivo por el que el resultado coincide o difiere de la predicción.

Transferencia

Elige la práctica P03 o P04. Explica qué cambiarías si el problema necesitara compartir ciertos objetos de manera intencional. Especifica cuáles compartirías y qué modificaciones serían observables desde cada referencia.

Mi explicación y mi decisión de diseño:

Puedes necesitar varios intentos. Escribe por qué tu respuesta cambió y pide apoyo señalando la instrucción o flecha que no logras explicar. El número de aciertos por sí solo no describe tu comprensión.

Conserva la guía como referencia para las siguientes etapas del curso. Esta etapa se centra en los objetos incorporados de Python y sus referencias.

CONSULTA

Glosario y fuentes

Término	Significado en esta guía
Objeto	Dato con tipo, valor e identidad.
Referencia	Relación que permite llegar a un objeto desde un nombre o un contenedor.
Alias	Otro nombre o referencia que llega al mismo objeto.
Mutación	Cambio del estado de un objeto existente.
Asignación	Indicar qué objeto señala un nombre; puede ser el mismo que antes u otro.
Reasignación	Volver a asignar un nombre; en los ejemplos puede pasar a señalar otro objeto.
Identidad	Lo que distingue a un objeto concreto. Se compara con <code>is</code> y permanece mientras el objeto existe.
Copia superficial	Nuevo contenedor exterior que reutiliza referencias a sus elementos.
Copia profunda	Copia recursiva que puede conservar alias internos y reutilizar inmutables.

Documentación para ampliar

1. Python: objetos, valores y tipos

Consulta para 1.1-1.3 y 1.10: mutabilidad, identidad y contenedores.

<https://docs.python.org/3/reference/datamodel.html#objects-values-and-types>

2. Python: asignación de argumentos

Consulta para 1.5: asociación de parámetros y uso de valores devueltos.

<https://docs.python.org/3/faq/programming.html#how-do-i-write-a-function-with-output-parameters-call-by-reference>

3. Python: copia superficial y profunda

Consulta para 1.6, 1.7 y 1.9: operaciones del módulo copy.

<https://docs.python.org/3/library/copy.html>

4. Python: operaciones de secuencias

Consulta para 1.8: repetición de referencias y construcción de listas.

<https://docs.python.org/3/library/stdtypes.html#common-sequence-operations>

5. Raspberry Pi Foundation: PRIMM

Referencia del diseño de actividades: predecir, ejecutar, investigar, modificar y crear.

https://static.raspberrypi.org/files/curriculum/quickreads/11-Pedagogy_Summary_PRIMM_US_PRIMM.pdf

Selección y adaptación: los casos de listas, funciones, matrices, copias y equipos parten del material de referencia aportado para este curso. Las explicaciones, la progresión, los ejercicios adicionales y los diagramas se elaboraron para esta guía. Las referencias externas son de consulta; no necesitas abrirlas para realizar las actividades.